SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE:** CSA1737

COURSE OUTCOME COVERED

CO3: *Analyze knowledge representation and reasoning using propositional logic, FOL, inference, resolution, chaining methods, knowledge representation to perform logical reasoning for drawing valid conclusions and decision making. (BL4)*

Assessment Tool Weightage: 50%

Time Duration: 1 Hour

QUESTIONS: 4 **Total Marks: 40**

Annexure A

CASE STUDY

Code of Conduct:

*I **Saai Vardhan D / 192525205** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Saai Vardhan D

Annexure A

CASE STUDY

Name: Saai Vardhan D | Reg. No.: 192525205 | Course: CSA1737 – Artificial Intelligence

Case Study 1 — Smart Medical Diagnosis System (10 Marks)

Given: A rule-based logical agent for preliminary diagnosis. Rules: (I) Fever ∧ Cough → Possible Flu; (II) Fever ∧ Rash → Possible Measles; (III) Cough ∧ Breathlessness → Possible Pneumonia. Facts about Patient A: Fever = True, Cough = True, Breathlessness = True.

(a) Propositional-logic representation (3 marks)

Symbol	Meaning (proposition is TRUE when …)
F	Patient has Fever
C	Patient has Cough
R	Patient has Rash
B	Patient has Breathlessness
Flu	Patient has possible Flu (diagnosis)
Meas	Patient has possible Measles
Pneu	Patient has possible Pneumonia

Item	Natural language	Propositional logic	Clause (Horn form)
Rule I	Fever AND Cough → Possible Flu	F ∧ C → Flu	¬F ∨ ¬C ∨ Flu
Rule II	Fever AND Rash → Possible Measles	F ∧ R → Meas	¬F ∨ ¬R ∨ Meas
Rule III	Cough AND Breathlessness → Possible Pneumonia	C ∧ B → Pneu	¬C ∨ ¬B ∨ Pneu
Fact 1–3	Patient A: Fever, Cough, Breathlessness are True	F = T, C = T, B = T	F; C; B (unit clauses)

KB = { F ∧ C → Flu, F ∧ R → Meas, C ∧ B → Pneu, F, C, B }. No information about Rash is given, so R is unknown; under the closed-world assumption of a rule-based system, R is treated as not established (False). All three rules are **definite (Horn) clauses**, so Forward and Backward Chaining are sound and complete for this KB.

(b) Forward chaining – all possible diagnoses for Patient A (4 marks)

Forward chaining is **data-driven**: starting from known facts it fires every rule whose premises are all satisfied (Modus Ponens: from $p_1 \wedge \ldots \wedge p_n \rightarrow q$ and $p_1 \ldots p_n$ infer q), adds the conclusion to the fact set and repeats until no new fact appears (fixed point). A counter stores the number of unproven premises of each rule; a rule fires when its counter reaches 0.

Step	Agenda / fact processed	Rule counters (unproven premises)	Rule fired	Fact derived
0	Initial facts in agenda: F, C, B	R-I = 2, R-II = 2, R-III = 2	—	—

Step	Agenda / fact processed	Rule counters (unproven premises)	Rule fired	Fact derived
1	Process F	R-I: 2→1; R-II: 2→1	None (counters > 0)	—
2	Process C	R-I: 1→0; R-III: 2→1	**R-I:** F ∧ C → Flu fires (Modus Ponens)	**Flu**
3	Process B	R-III: 1→0	**R-III:** C ∧ B → Pneu fires (Modus Ponens)	**Pneu**
4	Process Flu	No rule has Flu in its premises	None	—
5	Process Pneu	No rule has Pneu in its premises	None	—
6	Agenda empty – fixed point reached	R-II still = 1 (Rash unknown)	R-II never fires	Meas is NOT derived

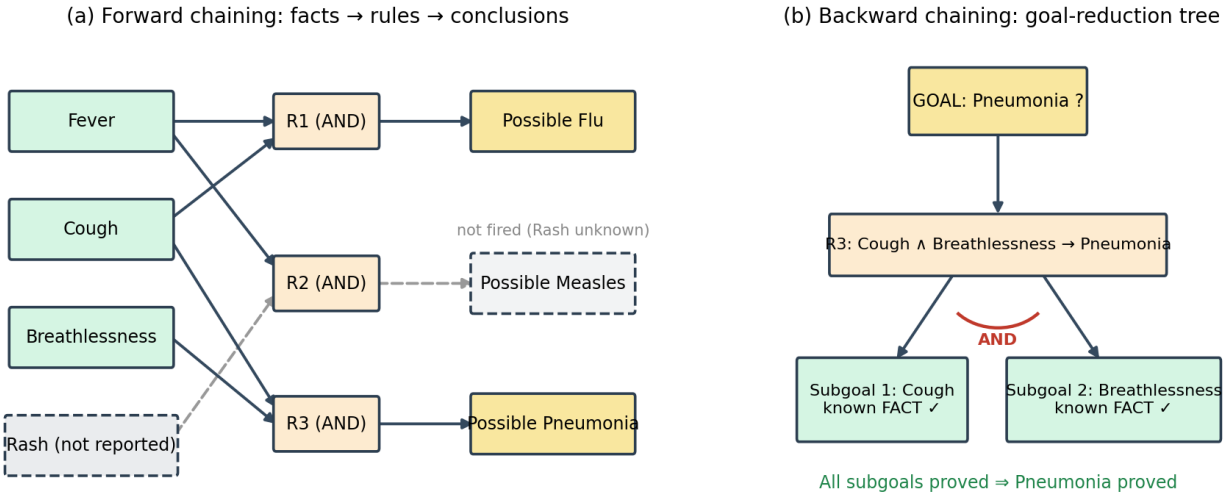
Inference chain: F, C ⊢ Flu (Rule I) and C, B ⊢ Pneu (Rule III). **Result:** Patient A has two possible diagnoses – **Possible Flu** and **Possible Pneumonia**. **Possible Measles is not supported**, because the premise Rash is absent from the facts, so Rule II cannot fire. (Because the diagnoses are only "possible", they should be confirmed by further tests.)

(c) Backward chaining – confirming "Patient A has Pneumonia" (3 marks)

Backward chaining is **goal-driven**: start from the goal, find rules whose conclusion matches it, and turn their premises into new subgoals (AND-sub-goals must all be proved; OR-alternatives need only one rule to succeed). A goal is proved if it is a known fact.

Step	Goal / subgoal being proved	Action	Result
1	Pneu ?	Not a known fact. Rules whose head is Pneu: only R-III (C ∧ B → Pneu). Replace goal by AND-subgoals {C, B}.	Goal reduced
2	C ? (Cough)	C is in the fact base (Fact 2).	Proved ✓
3	B ? (Breathlessness)	B is in the fact base (Fact 3).	Proved ✓
4	All subgoals of R-III proved	Conclusion of R-III holds (Modus Ponens).	**Pneu proved ✓ – goal confirmed**

Fig. 1 Case Study 1 – Forward chaining graph and backward-chaining goal tree



Comparison: forward chaining derived all consequences (Flu and Pneumonia) while backward chaining examined only the single rule relevant to the goal and never touched Rules I and II – it is more efficient when one specific hypothesis must be verified. Both agree that Pneumonia holds for Patient A.

Case Study 2 — Autonomous Traffic Management Agent (10 Marks)

Given (FOL KB): Rule 1: ∀x Vehicle(x) ∧ EmergencyType(x) → ClearPath(x). Rule 2: ∀x ClearPath(x) → GreenSignal(x) ∧ ¬RedSignal(x). Rule 3: ∀x ∀y Vehicle(x) ∧ Behind(x,y) ∧ GreenSignal(y) → Proceed(x). Facts: Vehicle(Ambulance), EmergencyType(Ambulance), Vehicle(CarA), Behind(CarA, Ambulance).

(a) Unification of Rule 1 with the facts – substitution sets (3 marks)

Unification finds a substitution θ that makes two literals identical: UNIFY(p, q) = θ ⇔ SUBST(θ, p) = SUBST(θ, q). The **Most General Unifier (MGU)** is the least restrictive such θ. Variables (x, y) may be replaced by constants/terms; two different constants (Ambulance, CarA) never unify. Premises of Rule 1: Vehicle(x), EmergencyType(x).

Step	Unify	Substitution / result
1	UNIFY(Vehicle(x), Vehicle(Ambulance))	θ_1 = {x/Ambulance} ✓
2	Apply θ_1 to next premise: EmergencyType(x)θ_1 = EmergencyType(Ambulance). UNIFY(EmergencyType(Ambulance), EmergencyType(Ambulance))	θ_2 = { } (already identical). Composite $\theta = \theta_1 \circ \theta_2$ = **{x/Ambulance}** ✓
3	Conclusion of Rule 1 under θ: ClearPath(x)θ	**ClearPath(Ambulance)** derived (Generalised Modus Ponens)
4	Alternative binding: UNIFY(Vehicle(x), Vehicle(CarA))	θ_1' = {x/CarA} ✓
5	Then EmergencyType(CarA) must match a fact: UNIFY(EmergencyType(CarA), EmergencyType(Ambulance))	FAILS – constants CarA ≠ Ambulance, and no fact EmergencyType(CarA) exists. So ClearPath(CarA) is NOT derived.

MGU for Rule 1 = θ = {x/Ambulance}. Continuing forward with this unifier:

Rule	Unification	Derived
Rule 2	UNIFY(ClearPath(x), ClearPath(Ambulance)) = {x/Ambulance}	GreenSignal(Ambulance) ∧ ¬RedSignal(Ambulance)
Rule 3	UNIFY of Vehicle(x), Behind(x,y), GreenSignal(y) with Vehicle(CarA), Behind(CarA,Ambulance), GreenSignal(Ambulance): x/CarA then y/Ambulance	θ = {x/CarA, y/Ambulance} ⇒ Proceed(CarA)

(b) Resolution proof of Proceed(CarA) (4 marks)

Step 1 – Convert to CNF (eliminate →, move ¬ inward, drop ∀ after standardising variables, distribute ∨ over ∧):

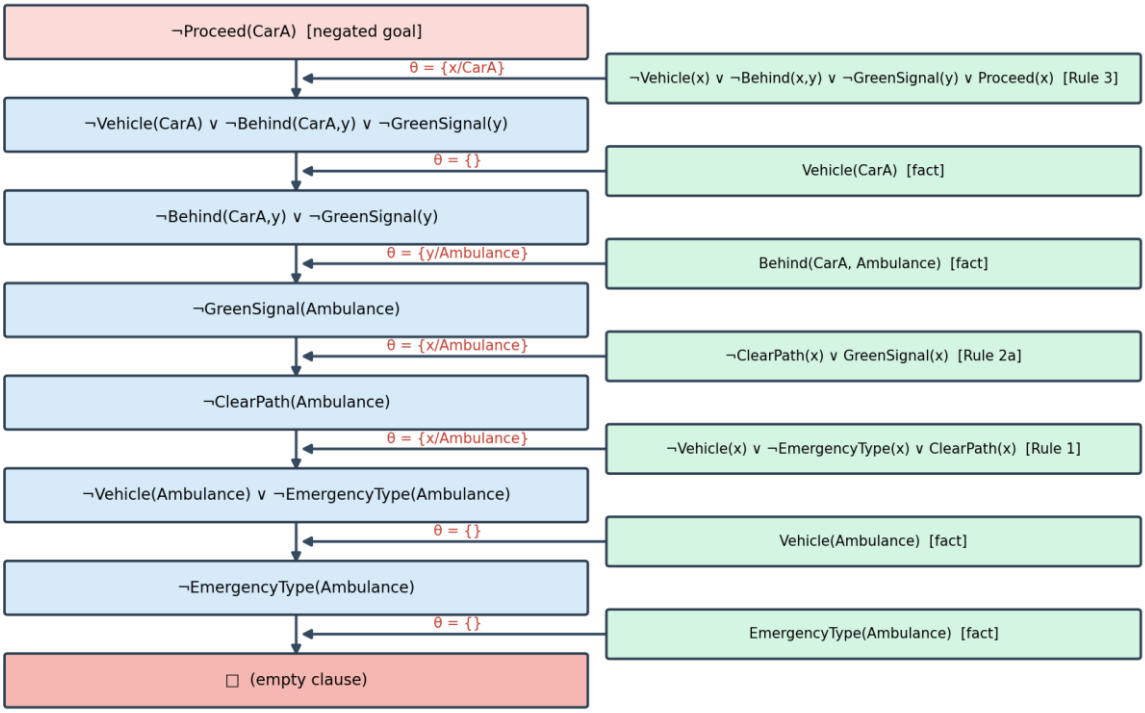
Clause	Source	CNF conversion
C1	Rule 1	¬(Vehicle(x) ∧ EmergencyType(x)) ∨ ClearPath(x) ⇒ ¬Vehicle(x) ∨ ¬EmergencyType(x) ∨ ClearPath(x)
C2a, C2b	Rule 2	¬ClearPath(x) ∨ (GreenSignal(x) ∧ ¬RedSignal(x)) ⇒ distribute ⇒ (¬ClearPath(x) ∨ GreenSignal(x)) ∧ (¬ClearPath(x) ∨ ¬RedSignal(x)). C2a: ¬ClearPath(x) ∨ GreenSignal(x); C2b: ¬ClearPath(x) ∨ ¬RedSignal(x)
C3	Rule 3	¬(Vehicle(x) ∧ Behind(x,y) ∧ GreenSignal(y)) ∨ Proceed(x) ⇒ ¬Vehicle(x) ∨ ¬Behind(x,y) ∨ ¬GreenSignal(y) ∨ Proceed(x)

Clause	Source	CNF conversion
C4–C7	Facts	C4: Vehicle(Ambulance); C5: EmergencyType(Ambulance); C6: Vehicle(CarA); C7: Behind(CarA, Ambulance)
C8	Negated goal	¬Proceed(CarA)

Step 2 – Resolution refutation (add ¬goal and derive the empty clause □):

Step	Clauses resolved	Unifier θ	Resolvent
1	C8 and C3 (Proceed(CarA) vs Proceed(x))	{x/CarA}	C9: ¬Vehicle(CarA) ∨ ¬Behind(CarA,y) ∨ ¬GreenSignal(y)
2	C9 and C6 (Vehicle(CarA))	{ }	C10: ¬Behind(CarA,y) ∨ ¬GreenSignal(y)
3	C10 and C7 (Behind(CarA,Ambulance))	{y/Ambulance}	C11: ¬GreenSignal(Ambulance)
4	C11 and C2a (GreenSignal(x))	{x/Ambulance}	C12: ¬ClearPath(Ambulance)
5	C12 and C1 (ClearPath(x))	{x/Ambulance}	C13: ¬Vehicle(Ambulance) ∨ ¬EmergencyType(Ambulance)
6	C13 and C4 (Vehicle(Ambulance))	{ }	C14: ¬EmergencyType(Ambulance)
7	C14 and C5 (EmergencyType(Ambulance))	{ }	□ **(empty clause)**

Fig. 2 Resolution refutation proof for Proceed(CarA) (blue = resolvent, green = clause from the KB)



The empty clause shows that KB ∧ ¬Proceed(CarA) is unsatisfiable, therefore **KB ⊨ Proceed(CarA)** – the proposition is proved. (Resolution is sound and refutation-complete for FOL.)

(c) Is the KB sufficient for two simultaneous emergency vehicles? (3 marks)

Conclusion: No – the KB is not sufficient. It is only safe when exactly one emergency vehicle needs a path. Suppose the facts also contain Vehicle(FireTruck) and EmergencyType(FireTruck), approaching the same junction from a crossing direction.

- By Rule 1 with $\theta = \{x/\text{FireTruck}\}$: ClearPath(FireTruck); by Rule 2: GreenSignal(FireTruck) ∧ ¬RedSignal(FireTruck). The KB therefore concludes GreenSignal(Ambulance) **and** GreenSignal(FireTruck) at the same time.
- In the real world, crossing directions cannot be green together. If we add the physical constraint "a green signal for one of two conflicting routes means red for the other" (Conflict(x,y) ∧ GreenSignal(x) → RedSignal(y)), we derive RedSignal(FireTruck) from the Ambulance's green, but Rule 2 already gave ¬RedSignal(FireTruck). The KB becomes **inconsistent**, and from an inconsistent KB every sentence is entailed (explosion) – the agent's conclusions become worthless.
- Rule 3 is also unsafe: a car behind either emergency vehicle may Proceed as soon as that vehicle has green, even if its route crosses the other emergency vehicle's path.

Missing element	Why it is needed (logical justification)	Additional rule / fact
Location, direction and junction	KB has no spatial information, so it cannot decide whether two vehicles conflict.	Facts: Approaching(x, junction, direction), e.g. Approaching(Ambulance, J1, North), Approaching(FireTruck, J1, East).
Conflict relation	Needed to detect simultaneous, incompatible green signals.	∀x∀y Approaching(x,j,d_1) ∧ Approaching(y,j,d_2) ∧ CrossingRoutes(d_1,d_2) → Conflict(x,y); Conflict is symmetric.
Mutual-exclusion (safety) axiom	Prevents the inconsistency above and makes the KB consistent.	∀x∀y Conflict(x,y) ∧ x ≠ y → ¬(GreenSignal(x) ∧ GreenSignal(y)).
Priority / arbitration	When two emergency vehicles conflict, exactly one must be served first; all others wait.	Priority(x,p) facts; ∀x∀y Conflict(x,y) ∧ HigherPriority(x,y) → Yield(y); tie-break: EarlierArrival(x,y) → HigherPriority(x,y).
Modified Rule 1 (non-monotonic guard)	Rule 1 grants ClearPath unconditionally; it must be conditional on not yielding.	∀x Vehicle(x) ∧ EmergencyType(x) ∧ ¬Yield(x) → ClearPath(x); Yield(y) → HoldSignal(y) ∧ RedSignal(y).
Non-conflicting vehicles	Efficiency: vehicles on compatible routes can both go.	¬Conflict(x,y) ∧ EmergencyType(x) ∧ EmergencyType(y) → both may receive ClearPath.
Rule 3 refinement	Followers must not enter conflicting paths.	Vehicle(x) ∧ Behind(x,y) ∧ GreenSignal(y) ∧ ¬∃z (EmergencyType(z) ∧ Conflict(x,z) ∧ GreenSignal(z)) → Proceed(x).
Unique-names / inequality	So that distinct vehicle constants are treated as different objects in x ≠ y.	Ambulance ≠ FireTruck ≠ CarA (unique-name axioms).

With these additions, the KB remains consistent, derives ClearPath for exactly one of two conflicting emergency vehicles (or for both when routes do not conflict), and keeps Proceed(CarA) provable only when it is safe – so the agent can handle simultaneous emergency vehicles.

Case Study 3 — Agricultural Expert Reasoning System (10 Marks)

Given (propositional KB): P1: Soil Dry → IrrigationNeeded. P2: IrrigationNeeded ∧ CropWheat → ApplyDripMethod. P3: ¬ApplyDripMethod ∧ CropWheat → CropAtRisk. Facts: SoilDry = True, CropWheat = True.

(a) Conversion to Conjunctive Normal Form (3 marks)

Procedure: (1) eliminate implication using α → β ≡ ¬α ∨ β; (2) move ¬ inward using De Morgan's laws ¬(α ∧ β) ≡ ¬α ∨ ¬β and double-negation ¬¬α ≡ α; (3) distribute ∨ over ∧ where a conjunction sits inside a disjunction.

Sentence	Step 1: eliminate →	Step 2: move ¬ inward	Step 3: distribute ∨ over ∧ → final CNF
P1: SoilDry → IrrigationNeeded	¬SoilDry ∨ IrrigationNeeded	No negation to move (already literals)	No ∧ inside ∨ → **¬SoilDry ∨ IrrigationNeeded**
P2: IrrigationNeeded ∧ CropWheat → ApplyDripMethod	¬(IrrigationNeeded ∧ CropWheat) ∨ ApplyDripMethod	De Morgan: (¬IrrigationNeeded ∨ ¬CropWheat) ∨ ApplyDripMethod	Already a single disjunction → **¬IrrigationNeeded ∨ ¬CropWheat ∨ ApplyDripMethod**
P3: ¬ApplyDripMethod ∧ CropWheat → CropAtRisk	¬(¬ApplyDripMethod ∧ CropWheat) ∨ CropAtRisk	De Morgan: (¬¬ApplyDripMethod ∨ ¬CropWheat) ∨ CropAtRisk; double negation: (ApplyDripMethod ∨ ¬CropWheat) ∨ CropAtRisk	Single disjunction → **ApplyDripMethod ∨ ¬CropWheat ∨ CropAtRisk**
Fact: SoilDry	—	—	**SoilDry** (unit clause)
Fact: CropWheat	—	—	**CropWheat** (unit clause)

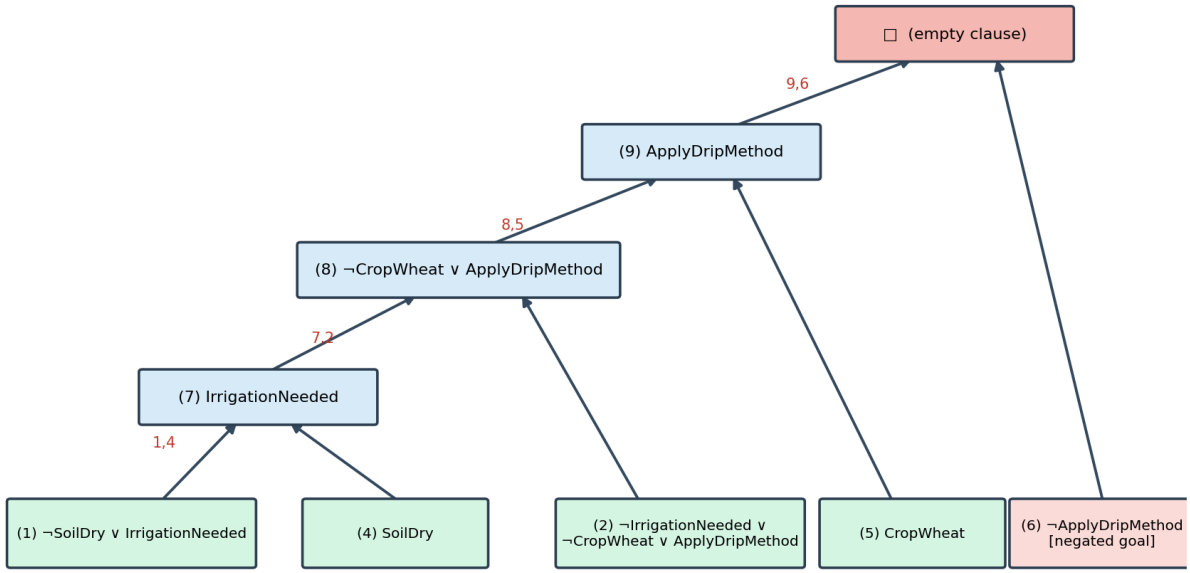
KB in CNF (clause set): (1) ¬SoilDry ∨ IrrigationNeeded (2) ¬IrrigationNeeded ∨ ¬CropWheat ∨ ApplyDripMethod (3) ApplyDripMethod ∨ ¬CropWheat ∨ CropAtRisk (4) SoilDry (5) CropWheat.

(b) Resolution refutation – proving ApplyDripMethod (4 marks)

To prove KB ⊨ ApplyDripMethod, add the **negated goal** (6) ¬ApplyDripMethod to the clause set and derive the empty clause □ (contradiction). Resolution rule: from (α ∨ p) and (β ∨ ¬p) infer (α ∨ β).

Step	Resolve	On literal	Resolvent
7	(1) ¬SoilDry ∨ IrrigationNeeded with (4) SoilDry	SoilDry	IrrigationNeeded
8	(7) IrrigationNeeded with (2) ¬IrrigationNeeded ∨ ¬CropWheat ∨ ApplyDripMethod	IrrigationNeeded	¬CropWheat ∨ ApplyDripMethod
9	(8) ¬CropWheat ∨ ApplyDripMethod with (5) CropWheat	CropWheat	ApplyDripMethod
10	(9) ApplyDripMethod with (6) ¬ApplyDripMethod	ApplyDripMethod	□ **(empty clause)**

Fig. 3 Resolution proof tree for ApplyDripMethod (numbers beside arrows = clauses resolved)



Deriving □ means that KB ∧ ¬ApplyDripMethod is unsatisfiable, so **KB ⊨ ApplyDripMethod**: the agent should apply the drip method. (Clause (3) was not needed in this proof.)

(c) Effect of removing the fact SoilDry (3 marks)

New KB′ = {(1) ¬SoilDry ∨ IrrigationNeeded, (2), (3), (5) CropWheat}. We analyse what can still be concluded.

Step	Reasoning	Conclusion
1	P1 is an implication SoilDry → IrrigationNeeded. Modus Ponens needs its premise SoilDry, which is now absent. (Modus Tollens would need ¬IrrigationNeeded, which is also not known.)	IrrigationNeeded can no longer be derived.
2	P2 needs IrrigationNeeded ∧ CropWheat. IrrigationNeeded is no longer available, so Modus Ponens on P2 cannot fire.	ApplyDripMethod can no longer be derived (the earlier proof used SoilDry in step 7).
3	P3 needs ¬ApplyDripMethod ∧ CropWheat. CropWheat is known, but ¬ApplyDripMethod is **not** known – failing to prove ApplyDripMethod is not the same as proving its negation.	P3 cannot fire under classical logic.
4	Resolution check for CropAtRisk: negate goal ¬CropAtRisk; resolve with (3): ApplyDripMethod ∨ ¬CropWheat; with (5): ApplyDripMethod. No clause contains ¬ApplyDripMethod or ¬IrrigationNeeded as a resolution partner for further steps – clause set is saturated without □.	KB′ ⊭ CropAtRisk.
5	Model-based confirmation: assign SoilDry = F, IrrigationNeeded = F, CropWheat = T, ApplyDripMethod = T, CropAtRisk = F. All clauses (1),(2),(3),(5) are true, but CropAtRisk is false.	A counter-model exists → CropAtRisk does not follow. Similarly the model with ApplyDripMethod = F, CropAtRisk = T shows ApplyDripMethod does not follow.

Conclusion	With SoilDry (original KB)	Without SoilDry (KB′)
IrrigationNeeded	Entailed (Modus Ponens, P1)	Not entailed
ApplyDripMethod	Entailed (P2 + resolution proof)	Not entailed

Conclusion	With SoilDry (original KB)	Without SoilDry (KB′)
CropAtRisk	Not entailed (¬ApplyDripMethod is false/unprovable)	Not entailed

Answer: No – CropAtRisk does not follow from KB′. Propositional entailment is **monotonic**: removing a fact can only shrink the set of conclusions, so IrrigationNeeded and ApplyDripMethod are lost, and the third rule cannot add CropAtRisk because its premise ¬ApplyDripMethod is not entailed.

Reasoning patterns used: Modus Ponens (forward derivation when the premise is present), Modus Tollens (checked, inapplicable), Resolution refutation (proof of non-entailment by saturation) and model checking (counter-model). **Note:** a rule-based expert system using the closed-world assumption / negation as failure would assume ¬ApplyDripMethod (since it cannot be proved) and then conclude CropAtRisk via P3 – a defeasible, non-monotonic default conclusion that is **not logically valid**. In practice the agent should flag this as a warning and collect more data (e.g. a soil-moisture reading) instead of asserting a certain conclusion.

Case Study 4 — Wumpus World Knowledge Agent (10 Marks)

Given: Percepts: Stench at [1,2], Breeze at [1,1], Glitter at [2,2]. Rules: (1) Stench[1,2] → Wumpus is adjacent to [1,2]; (2) Breeze[1,1] → Pit is adjacent to [1,1]; (3) Glitter[x,y] → Gold at [x,y]; (4) ¬Wumpus[x,y] ∧ ¬Pit[x,y] → Safe[x,y]. Cells are written [column, row]; adjacent cells share a side (grid 4 × 4, [1,1] bottom-left).

(a) Belief state and Modus Ponens (3 marks)

Adjacency needed: Adj[1,2] = {[1,1], [1,3], [2,2]} (cell [0,2] does not exist); Adj[1,1] = {[1,2], [2,1]}.

#	Percept / fact in the belief state	Rule used (Modus Ponens: p, p → q ⊢ q)	Derived knowledge
B0	Percepts: Stench[1,2], Breeze[1,1], Glitter[2,2]	—	Initial belief state
D1	Stench[1,2]	Rule 1: Stench[1,2] → Wumpus adjacent to [1,2]	Wumpus[1,1] ∨ Wumpus[1,3] ∨ Wumpus[2,2]
D2	Breeze[1,1]	Rule 2: Breeze[1,1] → Pit adjacent to [1,1]	Pit[1,2] ∨ Pit[2,1]
D3	Glitter[2,2]	Rule 3 with x = 2, y = 2	**Gold[2,2]**
D4	No ¬Wumpus and ¬Pit literals known for any cell	Rule 4 cannot fire yet	No cell is proved Safe from percepts alone

All derived conclusions: (i) exactly one Wumpus lies in {[1,1], [1,3], [2,2]}; (ii) at least one Pit lies in {[1,2], [2,1]}; (iii) the Gold is at [2,2]; (iv) no cell is yet proved Safe.

(b) Forward chaining – possible locations of Wumpus and Pit (4 marks)

Step	Agenda / fact	Rule fired	New fact(s) added
1	Stench[1,2]	Rule 1	Wumpus ∈ Adj[1,2] = {[1,1], [1,3], [2,2]}
2	Breeze[1,1]	Rule 2	Pit ∈ Adj[1,1] = {[1,2], [2,1]}
3	Glitter[2,2]	Rule 3	Gold[2,2]
4	Try Rule 4 for every cell	Needs ¬Wumpus ∧ ¬Pit	Not applicable – no cell has both literals known
5	Add background knowledge **B1**: the agent starts in [1,1] and is alive, so [1,1] holds neither Wumpus nor Pit	Rule 4 (¬Wumpus[1,1] ∧ ¬Pit[1,1])	Safe[1,1]; Wumpus[1,1] eliminated → Wumpus ∈ {[1,3], [2,2]}
6	Fixed point (using only the stated percepts and rules)	—	Wumpus ∈ {[1,3], [2,2]}; Pit ∈ {[1,2], [2,1]}

Possible locations: Wumpus at **[1,3] or [2,2]** (and [1,1] if the start cell were not known to be safe); Pit at **[1,2] or [2,1]**. Nothing more can be concluded from the percepts alone, so the candidates are shown with "?" in Fig. 4(a).

(c) Is the path [1,1] → [1,2] → [2,2] safe? (3 marks)

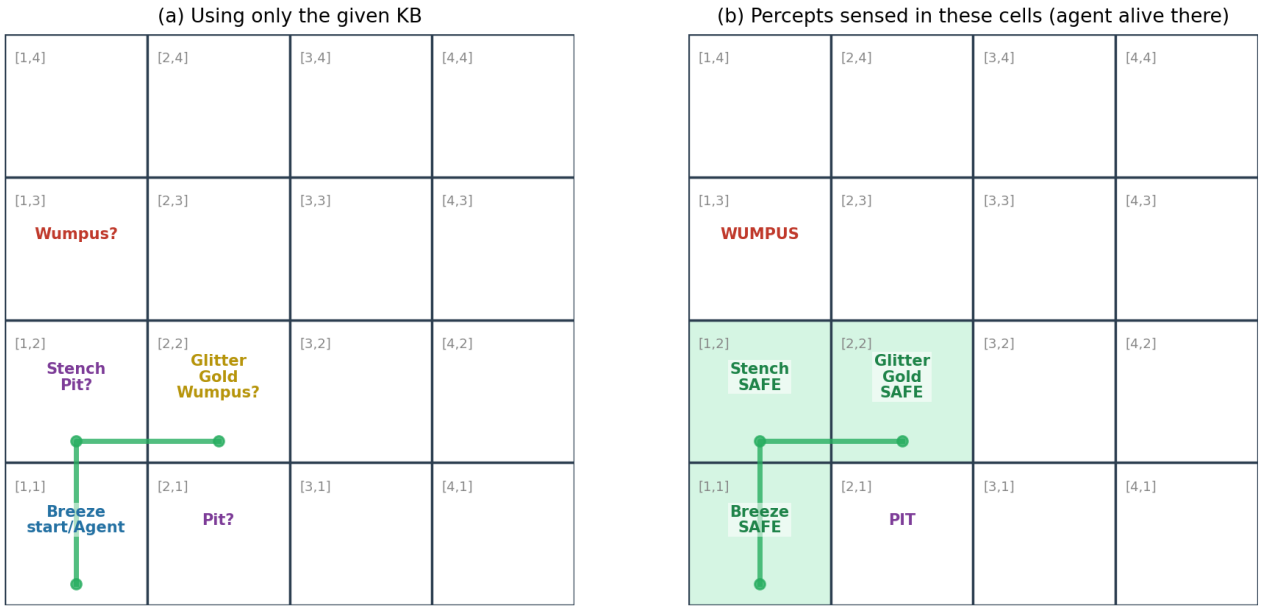
Cell on path	What the derived knowledge says	Safe provable?
[1,1]	Start cell – B1 gives ¬Wumpus ∧ ¬Pit ⇒ Safe[1,1] (Rule 4).	Yes
[1,2]	Stench here; Pit[1,2] ∨ Pit[2,1] is still open (from D2), so [1,2] might contain a Pit.	Not provable from the stated KB alone
[2,2]	Gold is here (D3), but the Wumpus may also be here (Wumpus ∈ {[1,3],[2,2]}).	Not provable from the stated KB alone

Evaluation 1 – strictly from the stated KB (rules 1–4 + percepts): the safety of [1,2] and [2,2] cannot be derived, because Rule 4 needs ¬Wumpus ∧ ¬Pit for those cells and the KB only says that a Pit may be at [1,2] and the Wumpus may be at [2,2]. A cautious logical agent must therefore **not claim the route is safe** until more evidence is collected (for example a no-breeze percept at [1,2]/[2,1]).

Evaluation 2 – using Wumpus World semantics: a percept is sensed only in the cell the agent is standing in. The stated percepts (Stench at [1,2], Glitter at [2,2], Breeze at [1,1]) therefore mean the agent has already been in those cells and is alive. Adding this background knowledge **B2** (a visited, surviving cell has no Wumpus and no Pit):

- [1,2] visited and alive ⇒ ¬Pit[1,2] ⇒ from D2 (disjunction elimination / resolution) Pit[2,1].
- [2,2] visited and alive ⇒ ¬Wumpus[2,2]; with ¬Wumpus[1,1] from B1, D1 gives Wumpus[1,3].
- Rule 4 then gives Safe[1,1], Safe[1,2] and Safe[2,2]. The hazards are at [1,3] (Wumpus) and [2,1] (Pit) – both **off the path** (Fig. 4(b)).

Fig. 4 Wumpus World – candidate hazards and the route [1,1] → [1,2] → [2,2] (green line)



Final conclusion: the route [1,1] → [1,2] → [2,2] is **safe** under the Wumpus-World convention that percepts are sensed at the agent's own cell – the agent can walk it, grab the Gold at [2,2] and return the same way, avoiding [2,1] and [1,3]. If instead the three percepts were only reported (the cells were never visited), the KB would not prove [1,2] and [2,2] safe, and the agent should gather further percepts before moving. This analysis uses Modus Ponens, disjunction elimination (unit resolution) and the safety rule 4.